Retrieval-augmented generation

Source: Wikipedia (en.wikipedia.org) — Educational Use

Retrieval-augmented generation (RAG) is a technique that enables large language models (LLMs) to retrieve and incorporate new information from external data sources. With RAG, LLMs first refer to a specified set of documents, then respond to user queries. These documents supplement information from the LLM's pre-existing training data. This allows LLMs to use domain-specific and/or updated information that is not available in the training data. For example, this enables LLM-based chatbots to access internal company data or generate responses based on authoritative sources. The technique was first proposed in 2020 and has since become a widely adopted approach in modern AI systems.

RAG improves LLMs by incorporating information retrieval before generating responses. Unlike LLMs that rely on static training data, RAG pulls relevant text from databases, uploaded documents, or web sources. According to Ars Technica, "RAG is a way of improving LLM performance, in essence by blending the LLM process with a web search or other document look-up process to help LLMs stick to the facts." This method helps reduce AI hallucinations, which have caused chatbots to describe policies that don't exist, or recommend nonexistent legal cases to lawyers that are looking for citations to support their arguments.

RAG also reduces the need to retrain LLMs with new data, saving on computational and financial costs. Beyond efficiency gains, RAG also allows LLMs to include sources in their responses, so users can verify the cited sources. This provides greater transparency, as users can cross-check retrieved content to ensure accuracy and relevance.

The term retrieval-augmented generation (RAG) was introduced in a 2020 paper that described combining a parametric language model with a non-parametric external memory accessed through retrieval at inference time.

RAG and LLM limitations

LLMs can provide incorrect information. For example, when Google first demonstrated its LLM tool "Bard" (later re-branded to Google Gemini), the LLM provided incorrect information about the James Webb Space Telescope. This error contributed to a $100 billion decline in Google's stock value. RAG is used to prevent these errors, but it does not solve all the problems. For example, LLMs can generate misinformation even when pulling from factually correct sources if they misinterpret the context. MIT Technology Review gives the example of an AI-generated response stating, "The United States has had one Muslim president, Barack Hussein Obama." The model retrieved this from the rhetorical chapter title "Barack Hussein Obama: America's First Muslim President?" in the book Faith in the New Millennium: The Future of Religion and American Politics. The LLM did not "know" or "understand" the context of the title, generating a false statement.

LLMs with RAG are programmed to prioritize new information. This technique has been called "prompt stuffing." Without prompt stuffing, the LLM's input is generated by a user; with prompt stuffing, additional relevant context is added to this input to guide the model's response. This approach provides the LLM with key information early in the prompt, encouraging it to prioritize the supplied data over pre-existing training knowledge.

Process

Retrieval-augmented generation (RAG) enhances large language models (LLMs) by incorporating an information-retrieval mechanism that allows models to access and utilize additional data beyond their original training set. Ars Technica notes that "when new information becomes available, rather than having to retrain the model, all that's needed is to augment the model's external knowledge base with the updated information ("augmentation"). IBM states that "in the generative phase, the LLM draws from the augmented prompt and its internal representation of its training data to synthesize" an answer.

RAG key stages

Typically, the data to be referenced is converted into LLM embeddings, numerical representations in the form of a large vector space. RAG can be used on unstructured (usually text), semi-structured, or structured data (for example knowledge graphs). These embeddings are then stored in a vector database to allow for document retrieval.

Given a user query, a document retriever is first called to select the most relevant documents that will be used to augment the query. This comparison can be done using a variety of methods, which depend in part on the type of indexing used.

The model feeds this relevant retrieved information into the LLM via prompt engineering of the user's original query. Newer implementations (as of 2023) can also incorporate specific augmentation modules with abilities such as expanding queries into multiple domains and using memory and self-improvement to learn from previous retrievals.

Finally, the LLM can generate output based on both the query and the retrieved documents. Some models incorporate extra steps to improve output, such as the re-ranking of retrieved information, context selection, and fine-tuning.

Applications

Retrieval-augmented generation is used in applications where generated responses need to be grounded in external or frequently updated information.

In healthcare, RAG has been studied as a way to ground large language model outputs in external medical knowledge sources, although reviews have noted continuing challenges around evaluation, ethics, and clinical reliability.

Improvements

Improvements to the basic process above can be applied at different stages in the RAG flow.

Encoder

These methods focus on the encoding of text as either dense or sparse vectors. Sparse vectors, which encode the identity of a word, are typically dictionary-length and contain mostly zeros. Dense vectors, which encode meaning, are more compact and contain fewer zeros. Various enhancements can improve the way similarities are calculated in the vector stores (databases).

Performance improves by optimizing how vector similarities are calculated. Dot products enhance similarity scoring, while approximate nearest neighbor (ANN) searches improve retrieval efficiency over K-nearest neighbors (KNN) searches.

Accuracy may be improved with Late Interactions, which allow the system to compare words more precisely after retrieval. This helps refine document ranking and improve search relevance.

Hybrid vector approaches may be used to combine dense vector representations with sparse one-hot vectors, taking advantage of the computational efficiency of sparse dot products over dense vector operations.

Other retrieval techniques focus on improving accuracy by refining how documents are selected. Some retrieval methods combine sparse representations, such as SPLADE, with query expansion strategies to improve search accuracy and recall.

Retriever-centric methods

These methods aim to enhance the quality of document retrieval in vector databases:

Pre-training the retriever using the Inverse Cloze Task (ICT), a technique that helps the model learn retrieval patterns by predicting masked text within documents.

Supervised retriever optimization aligns retrieval probabilities with the generator model's likelihood distribution. This involves retrieving the top-k vectors for a given prompt, scoring the generated response's perplexity, and minimizing KL divergence between the retriever's selections and the model's likelihoods to refine retrieval.

Reranking techniques can refine retriever performance by prioritizing the most relevant retrieved documents during training.

Language model

By redesigning the language model with the retriever in mind, a 25-time smaller network can get comparable perplexity as its much larger counterparts. Because it is trained from scratch, this method (Retro) incurs the high cost of training runs that the original RAG scheme avoided. The hypothesis is that by giving domain knowledge during training, Retro needs less focus on the domain and can devote its smaller weight resources only to language semantics. The redesigned language model is shown here.

It has been reported that Retro is not reproducible, so modifications were made to make it so. The more reproducible version is called Retro++ and includes in-context RAG.

Chunking

Chunking involves various strategies for breaking up the data into vectors so the retriever can find details in it.

Hybrid search

Sometimes vector database searches (aka semantic search technique) can miss key facts needed to answer a user's question. One way to mitigate this is to do a traditional text search (aka full text search), combine those results to the text chunks linked to the retrieved vectors from the vector search, and feed the combined hybrid text (using effective scoring or reranking scoring) into the language model for generation.

Challenges

RAG does not prevent hallucinations in LLMs. According to Ars Technica, "It is not a direct solution because the LLM can still hallucinate around the source material in its response."

While RAG improves the accuracy of large language models (LLMs), it does not eliminate all challenges. One limitation is that while RAG reduces the need for frequent model retraining, it does not remove it entirely. Additionally, LLMs may struggle to recognize when they lack sufficient

information to provide a reliable response. Without specific training, models may generate answers even when they should indicate uncertainty. According to IBM, this issue can arise when the model lacks the ability to assess its own knowledge limitations.

RAG poisoning

RAG systems may retrieve factually correct but misleading sources, leading to errors in interpretation. In some cases, an LLM may extract statements from a source without considering its context, resulting in an incorrect conclusion. Additionally, when faced with conflicting information, RAG models may struggle to determine which source is accurate. The worst case outcome of this limitation is that the model may combine details from multiple sources producing responses that merge outdated and updated information in a misleading manner. According to the MIT Technology Review, these issues occur because RAG systems may misinterpret the data they retrieve.

On September 2025, Israel paid Clock Tower X $6 million in order to sway public opinion to a more favourable view of the country (hasbara) after support plummeted due to its genocidal conduct during the Gaza war. Clock Tower X used RAG poisoning to flood the information space with sites and content sympathetic to Israel.

References